ViGoEmotions

Cẩm nang ba mô hình

Đọc mã nguồn • Hiểu quá trình học • Bảo vệ giữa kỳ

Tài liệu tiếng Việt dành cho sinh viên mới học ML/NLP.

Kết quả chính 05/11/2026: Hai notebook 08/09 executed và hai ZIP Kaggle equal20 đã hoàn tất. BiGRU/PhoBERT đều đủ 20 epoch ở cả 27/28 nhãn. PhoBERT Macro-F1 27 = 0,597410; Macro-F1 đủ 28 = 0,578341; bản 28 chiếu về 27 = 0,583580. Neutral F1 = 0,4369. Mọi bảng, đường học và kết luận trong cẩm nang dùng các lượt này. Test chưa chấm.

01 TF-IDF unigram/bigram → 27 hoặc 28 Logistic Regression theo OvR
02 Chuỗi token → embedding học từ đầu → BiGRU → 27/28 logits
03 VnCoreNLP → PhoBERT-base-v2 → full fine-tuning 20 epoch
04 Validation emotion27 Macro-F1 tại 0,5 → chọn best checkpoint

Nguồn: bằng chứng mới, lời dẫn/Q&A, profile tái lập. Thuật toán và ví dụ mã ở chương 1–6 giải thích nhánh 27; nhánh 28 dùng lại kiến trúc, thêm một output và chọn checkpoint trên 27 cột cảm xúc. Các default profile cũ xuất hiện trong đoạn mã nguyên văn không phải profile của notebook Kaggle: workflow truyền config equal20 rõ ràng.

Bạn cần trả lời được điều gì?

Một bình luận được biến thành số như thế nào? Mô hình học điều gì từ nhãn? Trọng số thay đổi ở dòng mã nào? Điểm validation nói được và chưa nói được điều gì?

Bản đối chiếu: 05/11/2026. Các đoạn trích mã được kiểm tra nguyên văn. Tất cả kết quả lấy từ evidence equal20, không lấy số model trong reports root hoặc các lượt ít epoch. Các notebook executed gốc giữ nguyên output.

Quy ước đọc bằng chứng

[MÃ] là hành vi có thể đọc trực tiếp trong repository. **[KẾT QUẢ]** là số đo có artifact hỗ trợ. **[GIẢI THÍCH]** là kiến thức hoặc diễn giải thiết kế. **[MINH HỌA]** là ví dụ tự tạo để học, không phải bình luận hay kết quả của dataset.

Mọi câu ví dụ trong tài liệu đều được tự tạo. Không trích bình luận gốc, từ vựng riêng của dataset, ID người dùng hoặc ca lỗi có thể khôi phục nội dung.

Mục lục và cách sử dụng

1. Hiểu bài toán trước khi học mô hình . . . 3

2. TF-IDF + One-vs-Rest Logistic Regression . . . 5

3. BiGRU học từ đầu . . . 8

4. PhoBERT-base-v2 và full fine-tuning . . . 11

5. Hiểu sâu vòng lặp huấn luyện . . . 14

6. Hiểu và tự tính các metric . . . 16

7. Giao thức 20 epoch và bằng chứng thí nghiệm . . . 18

8. So sánh ba mô hình một cách công bằng . . . 19

9. Chuẩn bị bảo vệ giữa kỳ . . . 21

10. Tờ tra cứu và nguồn đối chiếu . . . 22

Lộ trình ôn tập

Lượt 1 — hiểu ý chính: đọc Chương 1, sơ đồ đầu mỗi mô hình và bảng Chương 8. Hiểu bài toán trước khi thuộc tên lớp Python.

Lượt 2 — đọc mã: mở repository song song với Chương 2–5. Với mỗi đoạn mã, tự nói: đầu vào là gì, đầu ra có kích thước nào và dòng nào thực sự cập nhật tham số.

Lượt 3 — luyện bảo vệ: tự tính lại ví dụ Chương 6; nói bài ba phút; trả lời câu hỏi Chương 9 mà không đọc đáp án. Dùng tờ tra cứu cuối tài liệu trước buổi trình bày.

Tài liệu này có thể và không thể chứng minh

Các xác suất lưu tại reports/kaggle_equal20_2026-11-05/evidence/{27,28}/{model}/val_predictions.npz đã được tính lại metric và đối chiếu với CSV, thứ tự 27 nhãn, ID validation và checksum dữ liệu trong run JSON. Đây là xác minh artifact, không phải huấn luyện lại. Không suy ra khả năng tổng quát trên test, ý nghĩa thống kê hoặc quan hệ nhân quả của pretraining chỉ từ một lượt chạy.

Để tái tạo PDF sau khi sửa Markdown: .venv/bin/python scripts/build_study_guide.py. Cần reportlab, matplotlib, pypdf, pygments (bản dùng dựng: 5.0.1 / 3.11.2 / 6.20.1 / 2.21.0). Script kiểm tra trích mã nguyên văn; nếu mã nguồn đổi, cần cập nhật trích dẫn.

1. Hiểu bài toán trước khi học mô hình

1.1. NLP và phân loại cảm xúc

NLP là xử lý ngôn ngữ tự nhiên: giúp máy tính làm việc với ngôn ngữ con người. Ở đây, đầu vào là một bình luận tiếng Việt; đầu ra là những cảm xúc mà bình luận thể hiện. Đây là phân loại cảm xúc chi tiết, không chỉ chia thành tích cực và tiêu cực.

Multi-label nghĩa là một mẫu có thể mang nhiều nhãn. [MINH HỌA] “Mình vui và biết ơn vì được giúp đỡ” có thể chứa joy và gratitude. Hai cảm xúc cùng tồn tại; không cần ép máy chọn đúng một.

Hãy hình dung 27 công tắc trên một bảng điện: mỗi công tắc ứng với một cảm xúc. Một câu có thể bật nhiều công tắc hoặc không bật công tắc nào. Đây khác với một nút xoay chỉ được chọn một chế độ.

Bình luận tiếng Việt [MINH HỌA]: “Mình vui và biết ơn”
↓ Tiền xử lý và biểu diễn số theo từng mô hình
↓ 27 điểm số → 27 xác suất độc lập theo đầu ra
↓ So sánh mỗi xác suất với 0,5 → tập nhãn dự đoán

1.2. Vì sao 27 đầu ra? Neutral ở đâu?

[MÃ] scripts/common.py:16–20 liệt kê đúng 27 nhãn cảm xúc, ID 0–26. Dữ liệu còn có ID 27 cho Neutral. Đây là **không gian nhãn nguồn** 27 cảm xúc + Neutral; nhánh 27 có **27 đầu ra cảm xúc**, nhánh 28 thêm **Neutral làm đầu ra thứ 28**.

Nguồn [MÃ]: scripts/common.py:32–37.

```
def multihot(labels):
    # Project convention: neutral alone -> zero; mixed neutral -> retain emotions.
    result = np.zeros(27, dtype=np.int64)
    for label in set(parse_labels(labels)) - {27}:
        result[label] = 1
    return result
```

Dòng 34 tạo vector 27 số 0. Dòng 35 loại ID 27 và loại ID cảm xúc trùng nhau bằng phép tập hợp. Dòng 36 bật các vị trí cảm xúc. Dòng 37 trả kết quả. Vì thế [27] thành vector toàn 0; [2,9,27] giữ joy và gratitude.

Multi-hot là vector có thể có nhiều số 1. [MINH HỌA] Chỉ xét ba cột [joy, sadness, gratitude], câu trên có nhãn [1,0,1]. Trong nhánh 27 vector có 27 cột; nhánh 28 có 28 cột. Không nhầm multi-hot với one-hot chỉ có một số 1.

Dự đoán không có nhãn cảm xúc nghĩa là không đầu ra nào vượt ngưỡng. Có thể diễn giải như trạng thái không phát hiện cảm xúc theo quy ước dự án; nhánh 27 không học bộ phân loại Neutral riêng. Nhánh 28 giữ Neutral dương trong target và học trực tiếp; dự đoán rỗng vẫn không tự động là Neutral. Quy ước này không được mặc nhiên coi là giao thức của paper.

1.3. Sigmoid, softmax và cách chia dữ liệu

Sigmoid biến mỗi logit z thành xác suất p = 1/(1+exp(−z)). Mỗi đầu ra được xét riêng. [MINH HỌA] p(joy)=0,8 và p(gratitude)=0,7 cùng hợp lệ; tổng xác suất 27 nhãn không buộc bằng 1. Softmax buộc các đầu ra cạnh tranh để tổng bằng 1, thường phù hợp khi phải chọn một lớp trong các lớp loại trừ nhau.

“Độc lập” ở đây nói về quyết định nhị phân và hàm mất mát từng nhãn. Các cảm xúc thật vẫn có thể liên quan nhau; hai mô hình neural còn dùng chung biểu diễn trước lớp đầu ra.

Tập	Vai trò trong dự án
Train	16.530 mẫu sau loại trùng; học vocabulary/IDF và trọng số.
Validation	2.066 mẫu; đo metric, chọn epoch neural tốt nhất; phân tích ngưỡng thăm dò.
Test	2.067 mẫu; giữ cho đánh giá Final, không dùng metric để chọn baseline hay learning rate.

[KẾT QUẢ] Nguồn đếm: reports/tables/dataset_summary.csv, các dòng train/val/test. Train nguồn có 16.531 mẫu, loại 1 mẫu trùng văn bản chuẩn hóa với held-out nên train dùng thực tế là 16.530. Không chia lại dữ liệu.

1.4. Tiền xử lý chung và tránh data leakage

Nguồn [MÃ]: scripts/common.py:40–41.

```
def normalize(text):
    return re.sub(r'\s+', ' ', unicodedata.normalize('NFC', text)).strip()
```

[MÃ] normalize chuẩn hóa Unicode NFC, gộp mọi cụm khoảng trắng thành một dấu cách và bỏ khoảng trắng ở đầu/cuối. Hàm này không bỏ dấu tiếng Việt, không chuyển emoji thành chữ và không sửa teencode. Mỗi mô hình còn có cách tokenize riêng ở các chương sau.

Data leakage là thông tin đáng lẽ chưa được biết lại lọt vào bước học hoặc chọn mô hình, làm điểm đo lạc quan. Ví dụ: học IDF trên cả validation; xây vocabulary từ test; thử nhiều learning rate rồi chọn bằng test.

[MÃ] scripts/common.py:44–64, hàm load_data, kiểm tra schema, văn bản rỗng, ID trùng; loại train có văn bản NFC trùng val/test; báo lỗi nếu val và test trùng văn bản. Test được đọc để kiểm tra hợp đồng và trùng lặp, không đưa target test vào tối ưu trọng số hay chọn checkpoint. So trùng văn bản sau NFC không phát hiện mọi câu diễn đạt lại.

Train → fit đặc trưng / cập nhật trọng số
Validation → đo Macro-F1 / chọn checkpoint và cấu hình
Test → đánh giá sau khi chốt toàn bộ quyết định ở Final

[GIẢI THÍCH] Giữ split được cấp chưa chứng minh split có multilabel stratification. Paper chỉ nêu tỷ lệ 8:1:1 tại §5.1, trang PDF 6. docs/label_mapping.md ghi rõ giới hạn xác minh này.

2. TF-IDF + One-vs-Rest Logistic Regression

2.1. Từ văn bản thành vector số

Máy tính cần số để nhân ma trận và tối ưu hàm mất mát. TF-IDF biểu diễn bình luận bằng độ quan trọng của token/ngram trong tập văn bản, thay vì hiểu trực tiếp câu chữ như người đọc.

Unigram là một token. **Bigram** là hai token đứng cạnh nhau. [MINH HỌA] “không vui” tạo unigram không, vui và bigram không vui. Bigram giúp phân biệt một số cụm phủ định, nhưng không xử lý được mọi quan hệ ngữ nghĩa dài.

[MÃ] configs/kaggle_equal_20ep.json:22–31 dùng lowercase=true, token_pattern=(?u)\b\w+\b, ngram_range=[1,2]. Tokenizer regex nhận chuỗi ký tự từ Unicode, kể cả token một ký tự. Đây không phải tách từ VnCoreNLP; nhiều từ tiếng Việt gồm nhiều tiếng vẫn bị tách theo ranh giới regex. Emoji và dấu câu không thuộc \w nên không thành đặc trưng riêng ở cấu hình này. Không có bước loại stopword được cấu hình.

2.2. TF, IDF và ví dụ tính tay

TF đo token xuất hiện bao nhiêu lần trong một văn bản. Cấu hình dùng sublinear_tf=true: với số lần xuất hiện f > 0, TF = 1 + ln(f); nếu f = 0 thì trọng số bằng 0. Token lặp nhiều không tăng trọng số tuyến tính mãi.

IDF giảm trọng số token phổ biến trong nhiều tài liệu. Theo mặc định smooth_idf=True của scikit-learn: IDF(t) = ln((1+N)/(1+df(t))) + 1. N là số văn bản train; df(t) là số văn bản train chứa token t.

TF-IDF trước chuẩn hóa bằng TF × IDF. Mặc định norm='l2' chia vector cho độ dài Euclid. Các mặc định này không viết trong JSON; được đối chiếu với tài liệu TF-IDF scikit-learn.

[MINH HỌA] Ba tài liệu tự tạo: D1 “vui vui”, D2 “vui buồn”, D3 “buồn”. Chỉ giữ hai unigram để tính tay; đây không phải chạy đầy đủ ngram của dự án. N=3; cả hai token có df=2 nên IDF=ln(4/3)+1≈1,2877.

Văn bản	TF hai cột [vui, buồn]	TF-IDF trước L2	Sau L2
D1	[1,6931; 0]	[2,1802; 0]	[1; 0]
D2	[1; 1]	[1,2877; 1,2877]	[0,7071; 0,7071]
D3	[0; 1]	[0; 1,2877]	[0; 1]

Sparse vector chỉ lưu các phần tử khác 0 và vị trí của chúng. Một bình luận ngắn thường chỉ chứa rất ít trong hàng chục nghìn đặc trưng, nên biểu diễn thưa tiết kiệm bộ nhớ. [KẾT QUẢ] Vocabulary đã fit có 34.837 đặc trưng, không phải lúc nào cũng đủ mức trần 40.000 (reports/kaggle_equal20_2026-11-05/evidence/27/tfidf_lr/run.json, khóa features).

2.3. Đọc hàm huấn luyện thật, từng dòng

Không có train_baselines.py trong bản này. Điểm vào thực tế là scripts/train_tfidf.py::main.

Nguồn [MÃ]: scripts/train_tfidf.py:17–27.

```
    config = config or json.loads((ROOT/'configs/baseline27_4ep.json').read_text())
    out = ROOT/'reports' if output_dir is None else output_dir
    if output_dir is not None:
        out.mkdir(parents=True, exist_ok=False)
    frames, _ = load_data()
    params = config['tfidf'].copy()
    params['ngram_range'] = tuple(params['ngram_range'])
    vectorizer = TfidfVectorizer(**params)
    started = time.perf_counter()
    x = vectorizer.fit_transform(frames['train'].normalized)
    model = OneVsRestClassifier(LogisticRegression(**config['logistic_regression']), n_jobs=1)
```

Dòng 17: nhận config được truyền vào hoặc đọc profile baseline gốc. **18–20:** chọn namespace output; output riêng phải mới để không ghi đè. **21:** nhận các tập đã kiểm tra/lọc trùng. **22:** sao chép tham số TF-IDF. **23:** đổi danh sách [1,2] thành tuple (1,2).

24: khởi tạo TfidfVectorizer; chưa học vocabulary. **25:** bắt đầu đồng hồ. **26:** fit_transform vừa học vocabulary/IDF **chỉ từ train**, vừa chuyển train thành ma trận X thưa. **27:** tạo Logistic Regression từ config rồi bọc bằng OneVsRestClassifier, chạy từng nhãn với n_jobs=1.

Tham số	Giá trị và ý nghĩa
max_features	40.000: trần số đặc trưng được giữ.
ngram_range	(1,2): cả unigram lẫn bigram.
min_df	2: phải có trong ít nhất hai tài liệu train.
sublinear_tf, lowercase	Cùng là true: log TF và viết thường.
C	1,0: nghịch đảo cường độ regularization; C lớn thường phạt yếu hơn.
solver, max_iter	liblinear, 1.000: thuật toán tối ưu và giới hạn số vòng lặp solver.
random_state	42, trùng seed cấu hình chung.

Nguồn tham số: configs/kaggle_equal_20ep.json:22–38. Regularization L2 và class_weight=None là mặc định của Logistic Regression phiên bản 1.6.1 ghi trong run; JSON không bật cân bằng lớp tự động.

2.4. Logistic Regression và OvR là gì?

Dù tên có chữ Regression, ở đây thuật toán làm **phân loại**. Mỗi nhãn c có vector trọng số w và bias b: z_c = w_c·x + b_c; sigmoid biến z thành xác suất nhãn hiện diện. Huấn luyện tìm w,b để các nhãn thật có xác suất phù hợp, với hàm log-loss nhị phân và regularization. liblinear thực hiện tối ưu bên trong fit; không dùng vòng lặp AdamW của neural.

OvR trong multi-label có nghĩa: với mỗi cảm xúc, tạo bài toán “có cảm xúc này hay không”. Một mẫu mang ba cảm xúc là positive cho ba bộ phân loại và negative cho các bộ còn lại. Mỗi cột target multi-hot là một bài toán nhị phân; 27 cột dẫn đến 27 classifier. Không phải “nhãn này đối đầu với đúng một nhãn khác”.

Theo API OneVsRestClassifier 1.6.1, predict_proba ở multi-label trả xác suất biên từng nhãn; không chuẩn hóa cả hàng thành tổng 1. Phù hợp với nhiều công tắc có thể bật cùng lúc.

2.5. Fit, dự đoán và đánh giá

Nguồn [MÃ]: scripts/train_tfidf.py:28–35.

```
    with warnings.catch_warnings(), threadpool_limits(limits=1):
        warnings.simplefilter('error', ConvergenceWarning)
        model.fit(x, targets(frames['train']))
    seconds = time.perf_counter()-started
    val = model.predict_proba(vectorizer.transform(frames['val'].normalized))
    parameters = sum(est.coef_.size + est.intercept_.size for est in model.estimators_)
    metadata = {'parameter_count':parameters, 'train_seconds':seconds}
    save_evaluation('tfidf_lr',frames,{'val':val},metadata,output_dir=out)
```

28: giữ cảnh báo trong phạm vi fit và giới hạn thread của thư viện số xuống 1. **29:** biến ConvergenceWarning thành lỗi để không im lặng nhận mô hình chưa hội tụ trong hạn mức solver. **30:** fit toàn bộ OvR trên X và target (16.530,27).

31: dừng đồng hồ sau TF-IDF fit + classifier fit. **32:** chỉ transform validation bằng vocabulary/IDF đã học, rồi dự đoán xác suất; không fit lại. **33:** đếm hệ số và bias của các classifier. **34:** gom metadata. **35:** gửi xác suất validation cho save_evaluation.

[MÃ] Ngưỡng thực sự được áp dụng trong scripts/common.py:71–78: probabilities >= 0.5. Ví dụ tự tạo [0,7; 0,2; 0,5] thành [1;0;1]; dấu bằng được tính là có nhãn. Một hàng không có đầu ra ≥0,5 giữ vector toàn 0, không có fallback “lấy nhãn cao nhất”.

[KẾT QUẢ] 27×(34.837+1)=**940.626** tham số classifier, không gồm IDF. Macro-F1 validation **0,238825**, Micro-F1 **0,271238** tại ngưỡng 0,5 (reports/kaggle_equal20_2026-11-05/analysis/27_emotion27/tfidf_lr/tables/tfidf_lr_metrics.csv).

2.6. Ưu điểm, giới hạn và vai trò baseline

[GIẢI THÍCH] Mô hình dễ triển khai, làm việc hiệu quả với vector thưa; có thể xem hệ số của ngram để hiểu liên hệ với từng nhãn. Tuy nhiên repository không cung cấp một nghiên cứu hệ số, nên không tuyên bố đã tìm được những từ quyết định thực tế.

Nó phụ thuộc ngram quan sát được, không có embedding ngữ cảnh pretrained, khó nhận ra diễn đạt đồng nghĩa và quan hệ dài. OvR cũng không có cơ chế học trực tiếp tương tác đầu ra nhãn. Đây là mốc truyền thống hợp lý để kiểm tra các mô hình phức tạp có đem lại cải thiện hay không; đây là diễn giải thiết kế của nhóm, không phải trích lời paper.

> **Bài nói khoảng một phút:** “Mô hình đầu tiên biến bình luận thành vector TF-IDF của unigram và bigram. Vocabulary và IDF chỉ được học từ train để tránh leakage. Sau đó nhóm huấn luyện 27 Logistic Regression theo One-vs-Rest, mỗi mô hình trả lời có hay không một cảm xúc. Xác suất từng nhãn được so với ngưỡng 0,5 nên một câu có thể nhận nhiều nhãn. Cấu hình dùng tối đa 40.000 đặc trưng, thực tế có 34.837. Macro-F1 validation đạt khoảng 0,2388. Baseline này đơn giản và giúp kiểm tra lợi ích của việc học biểu diễn chuỗi và pretrained, nhưng khó hiểu ngữ cảnh dài hoặc từ ngữ khác cách viết đã gặp.”

Năm câu hỏi nhanh: (1) Vì sao dùng bigram? Để thêm tín hiệu cụm hai token, ví dụ phủ định. (2) Fit IDF ở đâu? Train, dòng 26. (3) Vì sao OvR? Vì target có 27 cột nhị phân. (4) Dùng AdamW không? Không, LR dùng solver liblinear. (5) Có đủ 40.000 feature không? Không, run lưu 34.837.

Ba ngộ nhận: tên Regression không có nghĩa dự đoán giá trị liên tục ở bài này; TF-IDF không phải embedding có ngữ cảnh; max_iter=1000 không phải 1.000 epoch neural.

3. BiGRU học từ đầu

3.1. Mạng neural, embedding và chuỗi

Mạng neural là chuỗi phép biến đổi có tham số học được. Trong mô hình này: bảng embedding biến token ID thành vector; GRU đọc chuỗi vector; head chuyển biểu diễn câu thành 27 logits. Trọng số được điều chỉnh từ lỗi dự đoán trên train.

Vocabulary là ánh xạ token → số nguyên. ID chỉ là địa chỉ tra bảng, không phải mức độ cảm xúc. **Embedding** là vector số thực tại địa chỉ đó; embedding của cùng token được dùng chung khi token xuất hiện ở nhiều vị trí. Trong BiGRU này, bảng embedding được khởi tạo mới và học cùng mạng, không nạp pretrained word vectors.

[MÃ] Token hóa BiGRU là NFC + lower + whitespace, tức chuẩn hóa chung, viết thường, rồi split() theo khoảng trắng. Không dùng VnCoreNLP. Ví dụ tự tạo “rất vui” tạo hai token; từ tiếng Việt nhiều tiếng có thể bị tách thành nhiều token.

PAD=0 dùng lấp cuối chuỗi ngắn. **UNK=1** đại diện token không có trong vocabulary. Token thật bắt đầu ID 2. Vocabulary chỉ được xây từ train nên gặp token mới ở validation là bình thường, không được thêm từ validation để né UNK.

3.2. Padding, truncation và kích thước tensor

Sequence length là số token trong chuỗi theo tokenizer của mô hình, không phải số ký tự. max_length=128: chuỗi dài bị cắt lấy 128 token đầu; chuỗi ngắn được đệm 0 tới 128.

[MINH HỌA] Vocabulary giả lập rất→2, vui→3, buồn→4; hai câu “rất vui” và “buồn lắm” thành [2,3], [4,1] nếu lắm chưa có trong từ điển. Để vẽ gọn, chỉ minh họa độ dài đệm 4 thay vì cấu hình thật 128.

Bước	Kích thước với B=2, L=4	Ý nghĩa
ids	(2,4)	[[2,3,0,0],[4,1,0,0]].
mask	(2,4)	[[1,1,0,0],[1,1,0,0]].
Embedding	(2,4,128)	Mỗi token thành vector 128 chiều.
hidden	(2,2,128)	2 hướng × batch 2 × hidden 128; GRU một tầng.
Ghép hai hướng	(2,256)	Mỗi câu có 256 đặc trưng.
Logits và sigmoid	(2,27)	27 điểm và 27 xác suất mỗi câu.

Với batch thật đủ 64 và L=128: ids là (64,128), embedding (64,128,128), hidden (2,64,128), logits (64,27). Batch cuối có thể nhỏ hơn 64.

3.3. GRU hai hướng và lớp đầu ra trong mã

GRU là mạng hồi quy có các cổng học được để giữ, thay và kết hợp thông tin qua từng bước chuỗi. **Hidden state** là vector trạng thái tóm tắt thông tin tại một bước. Không cần thuộc toàn bộ công thức cổng để giải thích: cổng giúp mạng điều chỉnh lượng thông tin cũ mang sang bước mới.

Bidirectional nghĩa là một GRU đọc xuôi và một GRU đọc ngược. Từ cùng câu, ta có hai tóm tắt theo hai chiều. Vì cả bình luận đã có sẵn trước khi phân loại, đọc ngược không dùng nhãn tương lai hay dữ liệu từ tập khác.

Nguồn [MÃ]: scripts/train_neural.py:19–31.

```
class BiGRU(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, dropout):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
        self.gru = nn.GRU(embedding_dim, hidden_dim, batch_first=True, bidirectional=True)
        self.dropout = nn.Dropout(dropout)
        self.head = nn.Linear(2*hidden_dim, 27)

    def forward(self, ids, mask):
        packed = pack_padded_sequence(self.embedding(ids), mask.sum(1).cpu(),
                                      batch_first=True, enforce_sorted=False)
        _, hidden = self.gru(packed)
        return self.head(self.dropout(torch.cat((hidden[-2],hidden[-1]),dim=1)))
```

19–21: khai báo mô hình và khởi tạo lớp cha. **22:** bảng embedding, ID 0 là padding nên hàng padding không nhận gradient học như token thường. **23:** GRU nhận embedding; batch_first=True đặt batch ở trục đầu; bidirectional=True bật hai hướng. num_layers không truyền nên mặc định một tầng.

24: dropout 0,3 khi train. **25:** head tuyến tính 256→27. **27–29:** embedding được đóng gói bằng pack_padded_sequence, độ dài thật tính từ tổng mask; PAD cuối câu không bị coi là dữ liệu thật. enforce_sorted=False cho phép batch không sắp theo độ dài.

30: lấy hidden cuối của GRU, không dùng toàn bộ output theo từng token. **31:** hidden[-2] là hướng xuôi và hidden[-1] hướng ngược của tầng cuối; ghép theo trục đặc trưng, áp dropout rồi head. Không dùng mean/max pooling. Hàm trả **logits**, sigmoid chỉ xuất hiện khi dự đoán.

3.4. Vocabulary và đổi token thành ID

Nguồn [MÃ]: scripts/train_neural.py:48–49.

```
        words = Counter(word for text in frames['train'].normalized for word in text.lower().split())
        vocab = {word:i+2 for i,(word,_) in enumerate(words.most_common(settings['max_vocab']-2))}
```

Dòng 48 đếm token chỉ trong train sau lowercase/whitespace. Dòng 49 lấy nhiều nhất max_vocab−2 token thường gặp, đánh số từ 2 để dành PAD/UNK. Tập huấn luyện thực tế tạo **15.025** mục gồm hai token đặc biệt (reports/kaggle_equal20_2026-11-05/evidence/27/gru/run.json, preprocessing.vocab_size), thấp hơn trần 30.000. scripts/train_neural.py:50–52 lưu vocabulary riêng trong data/derived/ và khởi tạo BiGRU; tài liệu này không công bố nội dung vocabulary.

Nguồn [MÃ]: scripts/train_neural.py:54–59.

```
            seqs = [[vocab.get(w,1) for w in text.lower().split()][:max_length]
                    for text in frames[split].normalized]
            ids = torch.zeros((len(seqs),max_length),dtype=torch.long)
            for i,seq in enumerate(seqs):
                ids[i,:len(seq)] = torch.tensor(seq,dtype=torch.long)
            features[split] = (ids,ids.ne(0).long())
```

54–55: token → ID, token lạ →1, cắt độ dài. **56:** cấp phát toàn số 0. **57–58:** chép chuỗi thật vào đầu hàng. **59:** mask bằng 1 tại mọi ID khác 0. Đây là cách giữ riêng PAD và UNK: UNK=1 vẫn là token có thật trong chuỗi.

3.5. Cấu hình và cách học

Thiết lập	Giá trị thực tế
Vocabulary tối đa / thực tế	30.000 / 15.025 (gồm PAD, UNK).
Embedding / hidden mỗi hướng	128 / 128; ghép hai hướng thành 256.
Dropout	0,3 ở sau khi ghép hidden, trước head.
Max length / batch size	128 token / 64 mẫu.
Epoch / learning rate	20 / 0,001.
Loss / optimizer	BCEWithLogitsLoss / AdamW.
Weight decay / gradient clipping	0,0 do fallback của code / norm tối đa 1,0.

Nguồn: configs/kaggle_equal_20ep.json:39–48; optimizer/loss/clipping ở scripts/train_neural.py:135–150. Đừng nhầm dropout ở đây với dropout bên trong nhiều tầng GRU: mã chỉ tạo nn.Dropout bên ngoài, không truyền tham số dropout vào nn.GRU.

[MÃ] prepare tạo TensorDataset(ids,mask,target_float32) rồi DataLoader, train có shuffle và val không shuffle (scripts/train_neural.py:81–85). Vòng lặp neural được dùng chung cho BiGRU và PhoBERT; Chương 4–5 trích đầy đủ đoạn cập nhật trọng số. Với BiGRU, logits gọi model(ids,mask) tại scripts/train_neural.py:89–92.

Nguồn [MÃ]: scripts/train_neural.py:95–102.

```
def predict(model, model_name, loader, device):
    model.eval()
    results = []
    with torch.no_grad():
        for ids,mask,_ in loader:
            output = logits(model,model_name,ids.to(device),mask.to(device))
            results.append(torch.sigmoid(output).cpu().numpy())
    return np.concatenate(results)
```

96: chuyển sang chế độ đánh giá, dropout bị tắt. **98:** không ghi graph gradient. **99–100:** đưa batch lên thiết bị, lấy logits. **101:** sigmoid từng logit, đưa xác suất về CPU/NumPy. **102:** nối batch thành ma trận xác suất toàn tập. Hàm không gọi backward hoặc optimizer.step.

[KẾT QUẢ] reports/kaggle_equal20_2026-11-05/evidence/27/gru/run.json có 20 epoch, checkpoint tốt nhất ở epoch 17; Macro-F1 validation **0,421976**, Micro-F1 **0,459037**. Không suy ra checkpoint 17 có nghĩa dừng ở 17; đó là số epoch cố định của thiết kế.

3.6. Vai trò và luyện nói

[GIẢI THÍCH] BiGRU là baseline neural ở giữa: có thứ tự chuỗi và biểu diễn học được, nhưng chưa có tri thức từ corpus pretraining bên ngoài. Nó giúp đặt câu hỏi học biểu diễn từ 16.530 mẫu train có ích đến đâu. Ưu điểm là kiến trúc gọn hơn PhoBERT; giới hạn là dữ liệu dự án hữu hạn, UNK và việc xử lý chuỗi hồi quy.

> **Bài nói khoảng một phút:** “BiGRU của nhóm được huấn luyện từ đầu. Nhóm xây vocabulary chỉ từ train, đổi token thành ID, đệm PAD và dùng UNK cho token lạ. Embedding 128 chiều được học cùng GRU. Hai hướng đọc xuôi và ngược tạo hai hidden state 128 chiều; ghép thành 256, qua dropout 0,3 và head 27 đầu ra. Mạng trả logits, dùng BCEWithLogitsLoss và AdamW với learning rate 0,001, batch 64 trong 20 epoch. Khi đánh giá, sigmoid chuyển logits thành xác suất. Macro-F1 validation tốt nhất khoảng 0,4220. Đây là phép so sánh với neural học biểu diễn từ dữ liệu dự án, chưa phải fine-tuning pretrained model.”

Năm câu hỏi: (1) Đã fine-tune BiGRU chưa? Không nạp pretrained nên là from scratch. (2) Vì sao hidden 256? Ghép hai hướng 128. (3) PAD có được đọc như từ không? Chuỗi được pack theo mask. (4) UNK khác PAD? UNK là token thật chưa biết, PAD là đệm. (5) Vocabulary lấy từ đâu? Chỉ train.

Ba ngộ nhận: token ID không phải embedding; hai hướng không phải hai mô hình phân loại riêng; embedding không tự có nghĩa tốt ngay trước khi học.

4. PhoBERT-base-v2 và full fine-tuning

4.1. Pretraining, Transformer và self-attention

Pretrained language model là mô hình đã học cấu trúc ngôn ngữ từ corpus lớn trước bài toán hiện tại. PhoBERT dựa trên RoBERTa, được pretrained cho tiếng Việt. **Pretraining** học biểu diễn ngôn ngữ tổng quát, chẳng hạn khôi phục token bị che; **fine-tuning** tiếp tục điều chỉnh tham số theo nhãn cảm xúc của dự án.

Transformer là kiến trúc dùng nhiều khối xử lý vector token và self-attention. Hiểu đơn giản: để tạo biểu diễn cho một token, self-attention học cách tổng hợp thông tin từ các token khác trong câu. Biểu diễn vì vậy có thể phụ thuộc ngữ cảnh. Đây là cơ chế tính toán, không phải bằng chứng mô hình đã hiểu cảm xúc như con người.

Theo model card PhoBERT-base-v2, bản v2 thuộc cỡ base, khoảng 135 triệu tham số, độ dài tối đa 256 token và có corpus pretraining mở rộng bằng OSCAR-2301 ngoài Wikipedia/News. Dùng mô hình tiếng Việt là lựa chọn hợp lý để tận dụng biểu diễn ngôn ngữ sẵn có; không bảo đảm phù hợp mọi teencode hay mỉa mai.

4.2. VnCoreNLP làm gì trong pipeline này?

Nguồn [MÃ]: scripts/train_neural.py:34–40.

```
def segmented_texts(frames, model_dir):
    import py_vncorenlp
    segmenter = py_vncorenlp.VnCoreNLP(annotators=['wseg'], save_dir=str(model_dir.resolve()))
    result = {}
    for split in ('train','val'):
        result[split] = [' '.join(segmenter.word_segment(text)) for text in frames[split].normalized]
    return result
```

35: import thư viện. **36:** nạp bộ tách từ với duy nhất annotator wseg. **37:** tạo kết quả. **38:** chỉ chạy train/val. **39:** nhận văn bản NFC, tách từ/câu và ghép các câu đã tách bằng dấu cách. **40:** trả văn bản cho tokenizer PhoBERT.

VnCoreNLP ở đây **không pretrain PhoBERT**, không học nhãn cảm xúc và không cập nhật trọng số mạng trong vòng lặp PyTorch. Nó là công cụ tiền xử lý tách từ. Các tiếng thuộc một từ có thể được nối bằng dấu gạch dưới. [MINH HỌA] “học sinh” có thể được biểu diễn thành học_sinh; đây không phải output đã chạy đo từ dataset.

Văn bản → NFC và gộp khoảng trắng
↓ VnCoreNLP wseg: xác định ranh giới từ tiếng Việt
↓ PhoBERT BPE: tách tiếp thành đơn vị con, đổi thành token ID
↓ Backbone pretrained → head cảm xúc → 27 logits

Tách từ và BPE là hai tầng khác nhau. BPE dùng vocabulary và quy tắc ghép đã học trước, có thể tách từ thành subword khi cần. Không huấn luyện lại tokenizer bằng train dự án. Hướng dẫn chính thức PhoBERT yêu cầu đầu vào đã tách từ; dự án làm theo pipeline đó. Không áp lowercase toàn bộ văn bản ở nhánh PhoBERT như BiGRU.

4.3. Nạp tokenizer, backbone và head 27 đầu ra

Nguồn [MÃ]: scripts/train_neural.py:64–70.

```
        from transformers import AutoTokenizer, AutoModelForSequenceClassification
        text = segmented_texts(frames,vncorenlp_dir)
        model_args = dict(revision=settings['revision'])
        tokenizer = AutoTokenizer.from_pretrained(settings['checkpoint'],use_fast=False,**model_args)
        model = AutoModelForSequenceClassification.from_pretrained(settings['checkpoint'],
            num_labels=27,problem_type='multi_label_classification',
            id2label=dict(enumerate(LABELS)),label2id={l:i for i,l in enumerate(LABELS)},**model_args)
```

64: dùng các lớp Auto chọn đúng loại tokenizer/model từ checkpoint. **65:** tách từ trước. **66:** giữ revision để cố định phiên bản pretrained. **67:** nạp tokenizer từ checkpoint, use_fast=False chọn triển khai tokenizer chậm của thư viện, không phải giảm learning rate.

68: nạp mô hình phân loại chuỗi có backbone pretrained. **69:** head có 27 đầu ra, khai báo bài toán multi-label. **70:** gắn mapping ID↔tên nhãn từ LABELS, cố định thứ tự cột. Head cảm xúc cho 27 nhãn được khởi tạo cho tác vụ, không phải tri thức cảm xúc ViGoEmotions có sẵn trong checkpoint ngôn ngữ.

Backbone là phần học biểu diễn ngôn ngữ. **Classification head** chuyển biểu diễn cấp chuỗi sang 27 logits. Repository dùng head do AutoModelForSequenceClassification cung cấp, không tự viết head PhoBERT trong train_neural.py. Vì logits(...) không truyền labels, loss được tính tường minh ở ngoài bằng BCEWithLogitsLoss, không lấy outputs.loss.

4.4. Token IDs, attention mask và DataLoader

Nguồn [MÃ]: scripts/train_neural.py:73–77.

```
            lengths = [len(tokenizer.encode(t, truncation=False)) for t in text[split]]
            truncation[split] = float(np.mean(np.array(lengths)>max_length))
            encoded = tokenizer(text[split],padding='max_length',truncation=True,
                                max_length=max_length,return_tensors='pt')
            features[split] = (encoded['input_ids'],encoded['attention_mask'])
```

73: đo độ dài token trước khi cắt để kiểm tra truncation. **74:** tỷ lệ chuỗi dài hơn giới hạn. **75–76:** tokenize, đệm tới max_length=256, cắt chuỗi dài, trả tensor PyTorch. Độ dài bao gồm token đặc biệt do tokenizer thêm; không đồng nhất với 256 từ tiếng Việt. **77:** lưu ID và mask. Attention mask cho biết vị trí hợp lệ và vị trí padding; không phải ma trận trọng số self-attention đã học.

Nguồn [MÃ]: scripts/train_neural.py:81–85.

```
    loaders = {}
    for split,(ids,mask) in features.items():
        dataset = TensorDataset(ids,mask,torch.tensor(targets(frames[split]),dtype=torch.float32))
        loaders[split] = DataLoader(dataset,batch_size=settings['batch_size'],
            shuffle=split=='train',num_workers=0,generator=torch.Generator().manual_seed(config['seed']))
```

81–82: tạo loader cho từng split. **83:** mỗi mẫu gồm IDs, mask và vector multi-hot float32. **84:** kích thước batch theo cấu hình. **85:** chỉ shuffle train, không tạo worker phụ; generator cố định seed. Với batch đủ 16: IDs/mask (16,256), target/logits (16,27). Khi dự đoán, scripts/train_neural.py:92 truyền input_ids và attention_mask, lấy .logits.

Thiết lập PhoBERT	Giá trị từ config
Checkpoint / revision	vinai/phobert-base-v2 / 86cd7fd4c148980922ac11a2cf5e257f2ba639e1.
Max length / batch / epoch	256 / 16 / 20.
AdamW learning rate / weight decay	0,00002 = 2e-5 / 0,01.
Loss / gradient clipping	BCEWithLogitsLoss / max norm 1,0.

Nguồn: configs/kaggle_equal_20ep.json:49–57, scripts/train_neural.py:135–150. Mã không tự đặt scheduler, warmup, gradient accumulation hoặc mixed precision; không thêm chúng vào phần mô tả thí nghiệm.

4.5. Fine-tuning thực sự xảy ra ở đâu?

Nguồn [MÃ]: scripts/train_neural.py:134–138.

```
    model.to(device)
    optimizer = torch.optim.AdamW(model.parameters(),lr=settings['learning_rate'],
                                  weight_decay=settings.get('weight_decay',0.0))
    loss_fn = nn.BCEWithLogitsLoss()
    history,best_score = [],-1.0
```

134: chuyển model lên GPU nếu có, ngược lại CPU. **135–136:** đưa **toàn bộ model.parameters()** cho AdamW, dùng learning rate và weight decay cấu hình. Nhánh PhoBERT không đặt requires_grad=False hay chỉ chọn model.classifier.parameters(). **137:** tạo BCE nhận logits. **138:** khởi tạo lịch sử và điểm tốt nhất.

Nguồn [MÃ]: scripts/train_neural.py:142–154.

```
    for epoch in range(1,settings['epochs']+1):
        model.train();total_loss = 0.0
        for ids,mask,y in loaders['train']:
            ids,mask,y = ids.to(device),mask.to(device),y.to(device)
            optimizer.zero_grad(set_to_none=True)
            loss = loss_fn(logits(model,args.model,ids,mask),y)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(),1.0)
            optimizer.step()
            total_loss += loss.item()*len(y)
        p = predict(model,args.model,loaders['val'],device)
        metrics = scores(targets(frames['val']),p)
        history.append(dict(epoch=epoch,loss=total_loss/len(frames['train']),**metrics))
```

142: lặp đủ số epoch. **143:** bật chế độ train và reset tổng loss thống kê. **144–145:** lấy batch, đưa dữ liệu lên thiết bị. **146:** xóa gradient cũ, không xóa trọng số. **147:** forward qua backbone và head, tính BCE với target multi-hot. **148:** backprop tính gradient cho graph. **149:** cắt norm gradient ở mức 1. **150:** AdamW cập nhật tham số. **151:** cộng loss theo số mẫu. **152–153:** dự đoán val và tính metric. **154:** lưu lịch sử epoch.

> **Vị trí fine-tuning: dòng 135–150.** Nạp pretrained chỉ là khởi đầu; backward() và optimizer.step() mới tạo quá trình học tiếp cho backbone và head.

4.6. Lưu và nạp checkpoint tốt nhất

Nguồn [MÃ]: scripts/train_neural.py:156–159.

```
        if metrics['macro_f1'] > best_score:
            best_score = metrics['macro_f1']
            torch.save(model.state_dict(),checkpoint)
            best_epoch = epoch
```

Điểm val tăng thì lưu state_dict; dấu > giữ epoch sớm khi hòa. Dòng 163 nạp best: model.load_state_dict(torch.load(checkpoint,map_location=device,weights_only=True)); sau đó đánh giá val (scripts/train_neural.py:163–166). Checkpoint mặc định: data/checkpoints/phobert_best.pt (scripts/train_neural.py:139). Tài liệu kiểm tra xác suất/lịch sử, chưa nạp trọng số riêng tư.

> **Bài nói khoảng một phút:** “Nhóm dùng PhoBERT-base-v2 đã pretrained cho tiếng Việt. Bình luận được chuẩn hóa, tách từ VnCoreNLP rồi BPE tokenize. Model có backbone pretrained và head 27 logits. Nhóm full fine-tuning: AdamW nhận toàn bộ tham số; mỗi batch tính BCEWithLogitsLoss, backward, clip gradient và step. Cấu hình là batch 16, learning rate 2e-5, weight decay 0,01, 20 epoch. Cuối mỗi epoch chỉ đo validation và lưu checkpoint khi Macro-F1 tăng. Baseline đạt Macro-F1 validation khoảng 0,5974; chưa kết luận learning rate 2e-5 tối ưu vì sweep ba mức mới được chuẩn bị.”

Năm câu hỏi: (1) VnCoreNLP học cảm xúc? Không, chỉ wseg. (2) Nạp pretrained đã là fine-tune? Chưa; cần học bằng nhãn tác vụ. (3) Freeze backbone? Giữ backbone cố định, chỉ head học. (4) Ai tính loss? Mã này tính BCE bên ngoài. (5) Logits là xác suất? Chưa; cần sigmoid.

Ba ngộ nhận: pretrained không đồng nghĩa đã học nhãn dự án; word segmentation không phải BPE; full fine-tuning không phải huấn luyện PhoBERT từ số 0.

5. Hiểu sâu vòng lặp huấn luyện

5.1. Batch, epoch và trọng số

Trọng số là các số mô hình học được: embedding, ma trận GRU, tham số Transformer, head và bias. **Hyperparameter** do người thiết kế chọn, ví dụ batch size hoặc learning rate; chúng không phải các trọng số được backward() trực tiếp học.

Batch size là số mẫu dùng chung cho một lần tính gradient. **Epoch** là một lượt duyệt hết train. [MÃ] DataLoader không đặt drop_last=True, nên giữ batch cuối chưa đủ kích thước (scripts/train_neural.py:83–85).

Với 16.530 mẫu train: BiGRU batch 64 có ceil(16.530/64)=259 batch/epoch; 258 batch đủ và batch cuối 18 mẫu. PhoBERT batch 16 có ceil(16.530/16)=1.034 batch/epoch; 1.033 batch đủ và batch cuối 2 mẫu. Đây là phép tính từ số mẫu/cấu hình, không phải đo thông lượng GPU.

Ở cấu hình hiện tại không có gradient accumulation và mỗi batch gọi optimizer.step() một lần: BiGRU 20 epoch có **5.180** lần step; PhoBERT 20 epoch có **20.680** lần step nếu chạy đủ không lỗi.

5.2. Dòng thời gian PhoBERT thực tế

Ký hiệu W0 là toàn bộ tham số lúc bắt đầu: backbone pretrained và head mới. W1, W2... là các bộ tham số sau từng lần step; đây là ký hiệu giải thích, không phải tên biến trong code.

EPOCH 1 — bắt đầu từ W0, batch size 16
Batch 1 → forward → BCE → backward → clip → AdamW → W1
Batch 2 → forward bằng W1 → BCE → backward → clip → AdamW → W2
... Batch 1.034 (2 mẫu) → W1034
Validation: eval + no_grad → sigmoid → Macro-F1 → lưu nếu tốt nhất
EPOCH 2 — tiếp tục từ W1034, không khởi tạo lại model
1.034 batch tiếp → W2068 → validation → xét checkpoint
EPOCH 3 — từ W2068 → W3102 → validation → xét checkpoint
EPOCH 4 — từ W3102 → W4136 → validation → xét checkpoint
... tiếp tục đến EPOCH 20 → W20680 → validation → xét checkpoint
Kết thúc: nạp checkpoint có Macro-F1 validation cao nhất

[MÃ] prepare(...) và optimizer được tạo **trước** vòng for epoch (scripts/train_neural.py:132–142). Vì vậy model và trạng thái AdamW tiếp tục từ batch/epoch trước. total_loss=0.0 chỉ reset biến thống kê; zero_grad chỉ xóa gradient. Không có dòng khởi tạo lại model trong mỗi epoch.

[KẾT QUẢ] PhoBERT 27 chạy đủ 20 epoch. Macro-F1 epoch 1 bằng 0; epoch 4 đạt 0,4339, best epoch 14 đạt 0,597410, epoch 20 đạt 0,5911. Bản 28 chọn epoch 15 theo emotion27 Macro-F1 = 0,583580, metric đủ 28 = 0,578341. Cuối lượt nạp best; không mặc định lấy epoch cuối. Nguồn là hai run JSON trong evidence equal20.

5.3. Forward, loss, gradient và AdamW

Forward pass dùng trọng số hiện tại để tính logits. **Loss** đo độ không phù hợp với target; thấp hơn nghĩa là dự đoán phù hợp target hơn theo hàm mất mát trên dữ liệu đang xét. **Backpropagation** dùng quy tắc đạo hàm để tính gradient: mỗi tham số ảnh hưởng loss theo hướng và độ lớn nào.

Với một nhãn, BCE=$-[y\ln(p)+(1-y)\ln(1-p)]$, p là sigmoid(z). Neural dùng BCEWithLogitsLoss nhận **z trực tiếp**, kết hợp sigmoid+BCE theo cách ổn định số. Không sigmoid rồi đưa vào BCEWithLogitsLoss lần nữa. Mặc định reduction lấy trung bình các phần tử batch×27; mã không bật pos_weight hay class weighting.

[MINH HỌA] Nhãn y=1: p=0,2 có loss $-\ln(0,2)\approx1,609$; p=0,8 có loss ≈0,223. Với y=0, p=0,8 bị phạt $-\ln(0,2)\approx1,609$. Đây là một nhãn, không phải loss trung bình thực nghiệm.

Optimizer dùng gradient để thay đổi trọng số. **Learning rate** điều chỉnh độ lớn bước cập nhật. Để hiểu hướng đi, ví dụ SGD giả lập $w_{mới}=w_{cũ}-lr\times gradient$: w=0,4, gradient=0,2, lr=0,001 cho w mới=0,3998. Đây không phải công thức cập nhật AdamW thực tế.

AdamW theo dõi trung bình động gradient và bình phương gradient để điều chỉnh bước theo từng tham số; còn có **weight decay** tách khỏi bước thích nghi, hướng tham số về nhỏ hơn. Trong code, PhoBERT dùng lr=2e-5 và decay=0,01; BiGRU lr=0,001 và decay=0,0 từ settings.get('weight_decay',0.0) (scripts/train_neural.py:135–136). Không dùng mặc định decay của thư viện để suy ra decay BiGRU.

Gradient clipping hạn chế norm chung của gradient, không cắt trực tiếp trọng số về [−1,1]. [MINH HỌA] Vector gradient [3,4] có norm 5; giới hạn 1 sẽ co về xấp xỉ [0,6;0,8]. Mã gọi clipping sau backward, trước step (scripts/train_neural.py:148–150). Clipping không phải early stopping.

5.4. Validation, checkpoint và điều kiện dừng

“Full” nghĩa là không đóng băng backbone; không bảo đảm mọi phần tử tham số đều thay đổi ở mọi batch.

Validation gọi model.eval() và torch.no_grad(); không gọi backward/step (scripts/train_neural.py:95–102). Nhãn val được dùng tính metric để **chọn** checkpoint, không trực tiếp tính gradient huấn luyện. eval() đổi hành vi dropout; tự nó không đồng nghĩa tắt autograd, nên code dùng thêm no_grad().

Checkpoint lưu trạng thái trọng số tại một thời điểm. Mã lưu model.state_dict(), không lưu optimizer state; đủ để nạp đánh giá tốt nhất, không phải một gói resume huấn luyện đầy đủ (scripts/train_neural.py:156–163).

Fixed epochs: chạy đủ 20 epoch cho cả BiGRU và PhoBERT ở cả hai bộ nhãn. **Early stopping:** dừng sớm theo quy tắc như không cải thiện sau k epoch; mã hiện tại không có quy tắc đó. Checkpoint tốt nhất có thể ở giữa vì học tiếp có thể giảm chất lượng val. Trong quá trình train, epoch sau vẫn bắt đầu từ trọng số cuối epoch trước; chỉ nạp best sau toàn bộ vòng lặp.

> **0,5 là ngưỡng giải mã xác suất, không phải lệnh dừng cập nhật trọng số.** Mạng tối ưu BCE trên logits của mọi batch, dù một xác suất đã vượt 0,5. F1 tại ngưỡng 0,5 dùng đánh giá/chọn checkpoint, không phải loss để đạo hàm trong code.

6. Hiểu và tự tính các metric

6.1. Một nhãn có bốn trường hợp

Xét riêng nhãn joy. **TP**: thật có, dự đoán có. **FP**: thật không, dự đoán có. **FN**: thật có, dự đoán không. **TN**: thật không, dự đoán không. Với multi-label, cùng một bình luận có thể là TP ở nhãn này và FN ở nhãn khác.

Nhãn joy [MINH HỌA]	Dự đoán có (1)	Dự đoán không (0)
Thật có (1)	TP = 2	FN = 0
Thật không (0)	FP = 1	TN = 1

Precision=TP/(TP+FP): trong những lần báo có cảm xúc, đúng bao nhiêu? **Recall**=TP/(TP+FN): trong những mẫu thật có cảm xúc, tìm ra bao nhiêu? **F1**=2PR/(P+R)=2TP/(2TP+FP+FN): cân bằng precision và recall. TN không xuất hiện trong công thức F1.

6.2. Ví dụ đủ ba nhãn, bốn mẫu

Tất cả dữ liệu dưới đây được tự tạo. Chỉ xét [joy, sadness, gratitude] để tính tay; dự án thật tính trên 27 nhãn.

Mẫu	Nhãn thật [J,S,G]	Xác suất [J,S,G]	Dự đoán tại ≥0,5
A	[1,0,1]	[0,8;0,2;0,4]	[1,0,0]
B	[0,1,0]	[0,6;0,7;0,1]	[1,1,0]
C	[1,1,0]	[0,7;0,3;0,2]	[1,0,0]
D	[0,0,1]	[0,1;0,2;0,8]	[0,0,1]

Nhãn	TP / FP / FN / TN	Precision	Recall	F1
Joy	2 / 1 / 0 / 1	2/3	1	4/5 = 0,8
Sadness	1 / 0 / 1 / 2	1	1/2	2/3 ≈ 0,6667
Gratitude	1 / 0 / 1 / 2	1	1/2	2/3 ≈ 0,6667

Hai ma trận còn lại đều có hàng thật có [TP=1,FN=1] và hàng thật không [FP=0,TN=2]. Mỗi ma trận đếm đúng 4 mẫu; không gộp các cảm xúc thành một bài toán chọn một lớp.

Macro-F1 là trung bình F1 từng nhãn: (0,8+2/3+2/3)/3 = 32/45 ≈ 0,7111. Không phải tính trung bình precision/recall trước rồi lấy F1; hai cách nói chung khác nhau.

Micro-F1: gộp TP=4, FP=1, FN=2 trên tất cả nhãn; 2×4/(2×4+1+2)=8/11≈0,7273. **Hamming Loss:** có 3 quyết định sai trong 4×3=12 quyết định, nên 3/12=0,25; càng thấp càng tốt.

6.3. Macro, Micro, Accuracy và mất cân bằng

Macro cho mỗi nhãn một phiếu bằng nhau, kể cả nhãn ít gặp. Micro cộng toàn bộ quyết định nên nhãn có nhiều mẫu dương ảnh hưởng mạnh hơn. Khi các cảm xúc mất cân bằng, Macro-F1 giúp nhìn thấy nhãn bị bỏ quên. Macro cao không bảo đảm mọi nhãn đều tốt; cần xem bảng per-label.

Macro-F1 **không phải accuracy**. Ở ví dụ trang trước, **subset accuracy** chỉ có mẫu D đúng trọn vẹn ba nhãn, nên 1/4=0,25; **label-wise accuracy** là 9/12=0,75; Macro-F1 là 0,7111. Khi phần lớn cặp mẫu–nhãn là 0, chỉ nhìn accuracy/Hamming có thể che giấu việc bỏ sót nhãn dương. TN nhiều không làm F1 tăng trực tiếp.

[MÃ] Chính sách khi không có mẫu số hợp lệ được đặt bằng zero_division=0: trường hợp F1 không xác định được gán 0, thay vì bỏ nhãn khỏi trung bình theo ý người viết báo cáo.

6.4. LRAP: xếp nhãn đúng lên cao

Label Ranking Average Precision (LRAP) đánh giá thứ tự xác suất: các nhãn thật có được xếp cao hơn nhãn không đúng không? Không cần ngưỡng 0,5. Với mỗi nhãn đúng, tính tỷ lệ nhãn đúng trong nhóm có score ≥score nhãn đó, rồi trung bình trên nhãn đúng và trên mẫu.

[MINH HỌA] Nhãn đúng J,G; điểm J=0,9, S=0,8, G=0,7: J có precision ở hạng của nó là 1/1; G là 2/3. LRAP mẫu=(1+2/3)/2=5/6. Ở bảng bốn mẫu trang trước: A=1; B=1; C=1; D=1, nên LRAP=1 dù Hamming=0,25. Nhãn đúng vẫn xếp trước nhãn sai, nhưng vài xác suất chưa vượt ngưỡng. Vì vậy LRAP và F1 trả lời câu hỏi khác nhau.

[GIẢI THÍCH THƯ VIỆN] scikit-learn xử lý mẫu không có nhãn dương với LRAP=1. Đã kiểm tra bằng ví dụ vector toàn 0; cần nhớ quy ước này vì Neutral-only thành toàn 0. LRAP cao không tự chứng minh phát hiện Neutral tốt. Xem API LRAP 1.6.1.

6.5. Nối mọi metric với mã

Nguồn [MÃ]: scripts/common.py:71–78.

```
def scores(y, probabilities, threshold=0.5):
    pred = probabilities >= threshold
    return dict(macro_f1=float(f1_score(y, pred, average='macro', zero_division=0)),
                micro_f1=float(f1_score(y, pred, average='micro', zero_division=0)),
                hamming_loss=float(hamming_loss(y, pred)),
                # Accumulate ranking precision in float64 across sklearn/SciPy versions.
                # Casting preserves the ordering/ties of saved float32 probabilities.
                lrap=float(label_ranking_average_precision_score(y, np.asarray(probabilities, dtype=np.float64))))
```

72: chuyển xác suất thành boolean bằng ≥threshold, mặc định 0,5. **73:** tính trung bình F1 trên 27 nhãn. **74:** tính F1 gộp. **75:** tỷ lệ cặp mẫu–nhãn sai. **76–78:** LRAP dùng xác suất chưa threshold, ép float64 để ổn định tính toán thứ hạng giữa các phiên bản thư viện.

save_evaluation ghi bảng metric tại 0,5 và precision/recall/F1/support từng nhãn (scripts/evaluate.py:31–41). Threshold sweep 0,05→0,80, bước 0,05 chỉ phân tích validation (scripts/evaluate.py:43–53). Đây là giải mã lại cùng xác suất, không huấn luyện thêm và không đổi điểm baseline ở ngưỡng 0,5.

7. Giao thức 20 epoch và bằng chứng thí nghiệm

7.1. Bốn việc cần phân biệt

Pretraining là học trước trên dữ liệu lớn; fine-tuning là cập nhật backbone/head trên bài toán này; chọn checkpoint là chọn epoch bằng validation; tuning hyperparameter là so sánh các cấu hình khác nhau. Hai lượt hiện tại fine-tune PhoBERT ở lr=2e-5 trong 20 epoch, không phải sweep nhiều learning rate.

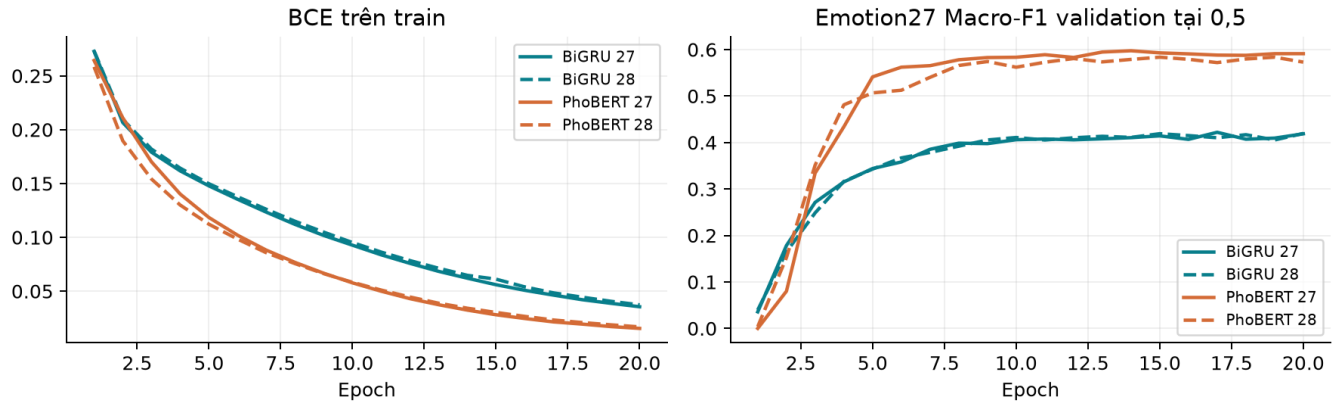
7.2. Cùng ngân sách, cùng cách chọn checkpoint

BiGRU/PhoBERT đều 20 epoch ở 27/28 nhãn; TF-IDF giữ liblinear/C=1/max_iter=1000/seed 42, không có epoch neural tương đương. Profile configs/kaggle_equal_20ep.json được truyền vào workflow. Giữ cùng source, split, normalization, tokenization và settings từng model; đầu ra 28 thêm Neutral. Best chọn theo emotion27 Macro-F1 tại 0,5; hòa lấy epoch sớm nhất. Epoch sau vẫn tiếp tục từ epoch trước; không early stopping.

7.3. Kiểm chứng nguồn và LF/CRLF

Hai notebook executed có 12 code cell mỗi bản, không error. Hai ZIP có đủ ba model; importer tính lại metric từ NPZ, kiểm tra IDs/targets/mapping/range, config, đủ history 20, best và checkpoint member. Hash source/config/CSV của bản 28 khác vì CRLF: đã so khớp biến thể byte LF/CRLF chính xác với local, không sửa manifest gốc. Config sau parse và fixed IDs cùng nhau; hai manifest cùng Tesla T4 và package versions. Một seed và session riêng chưa đo biến thiên.

7.4. Đường học mới



History 20 epoch của BiGRU/PhoBERT ở 27/28; F1 dùng cùng 27 cảm xúc, không phải sweep learning rate.

PhoBERT 27 best epoch 14, 28 best 15; BiGRU best 17 và 20. Train BCE giảm trong khi F1 validation có dao động. Chưa có validation loss; không khẳng định hội tụ hoặc overfitting chỉ từ loss train. Model có thể đạt best trước epoch 20 nhưng vẫn chạy hết ngân sách.

7.5. Ngưỡng là một phần decoder

Bảng chính giữ 0,5. Khảo sát hậu nghiệm từ NPZ mới cho thấy PhoBERT 27 đạt Macro-F1 0,597843 tại ngưỡng 0,35 so với 0,597410 tại 0,5; Hamming Loss tăng từ 0,055000 lên 0,058747. Đây là so sánh trên validation đã dùng lựa chọn, không phải cải thiện test hoặc lý do thay checkpoint. Final sẽ khóa một backbone rồi so fixed/global/per-label decoding trên validation trước khi chấm test.

8. So sánh ba mô hình một cách công bằng

8.1. Biểu diễn và ngân sách

Thành phần	TF-IDF + LR	BiGRU	PhoBERT
Biểu diễn	Unigram/bigram thưa; fit train	Embedding từ đầu	Backbone pretrained + VnCoreNLP/BPE
Phân loại	OvR Logistic Regression	Hai hướng GRU, ghép hidden, linear	Transformer + head logits
Ngân sách	max_iter=1000, không epoch neural	20 epoch, batch 64, lr=0,001	20 epoch, batch 16, lr=2e-5
Output	27/28 classifier	27/28 logits + sigmoid	27/28 logits + sigmoid

Neural dùng BCEWithLogitsLoss và AdamW, clip norm 1. Tất cả cùng threshold 0,5, seed 42, cùng dữ liệu; checkpoint neural chọn trên 27 cảm xúc. Cùng epoch không đồng nghĩa cùng số update hoặc compute.

8.2. Kết quả 27 cảm xúc

model	macro_f1	micro_f1	hamming_loss	lrap
TF-IDF + OvR LR	0.2388	0.2712	0.0606	0.6407
BiGRU	0.4220	0.4590	0.0626	0.6191
PhoBERT	0.5974	0.5996	0.0550	0.7419

8.3. Kết quả 28 nhãn có Neutral

model	macro_f1	micro_f1	hamming_loss	lrap
TF-IDF + OvR LR	0.2310	0.2662	0.0601	0.6096
BiGRU	0.4086	0.4483	0.0628	0.5851
PhoBERT	0.5783	0.5829	0.0575	0.7077

PhoBERT tốt nhất trong từng study theo bốn metric tổng hợp; xem phụ lục report để đọc từng nhãn. Macro-F1 27 và đủ 28 có mẫu số khác nhau, không dùng chênh trực tiếp để kết luận hiệu ứng Neutral.

8.4. So sánh cùng 27 cảm xúc và Neutral riêng

model	macro27	macro28_projected27	delta_pp	hamming27	hamming28_projected27
TF-IDF + OvR LR	0.2388	0.2388	0.0000	0.0606	0.0606
BiGRU	0.4220	0.4195	-0.2490	0.0626	0.0629
PhoBERT	0.5974	0.5836	-1.3830	0.0550	0.0575

Kết luận hiện tại: TF-IDF không đổi, BiGRU giảm khoảng 0,249 điểm phần trăm và PhoBERT giảm khoảng 1,383 điểm khi thêm Neutral rồi chấm trên cùng 27 cảm xúc. Thí nghiệm một seed này chưa cho thấy lợi ích trên 27 cảm xúc; không kết luận nhân quả hoặc ý nghĩa thống kê.

model	precision	recall	f1	support	TP	FP	FN
TF-IDF + OvR LR	0.5000	0.0106	0.0208	94	1	1	93
BiGRU	0.1778	0.0851	0.1151	94	8	37	86
PhoBERT	0.4018	0.4787	0.4369	94	45	67	49

Neutral được học trực tiếp trong bản 28 nhưng còn khó. Model 27 không có Neutral output, nên metric Neutral của nó không xác định.

8.5. Best epoch và chi phí

model	label_count	epochs_trained	best_epoch	train_seconds
TF-IDF + OvR LR	27	—	—	2.7524
BiGRU	27	20.0000	17.0000	80.3307
PhoBERT	27	20.0000	14.0000	14589.6182
TF-IDF + OvR LR	28	—	—	2.5247
BiGRU	28	20.0000	20.0000	83.6053
PhoBERT	28	20.0000	15.0000	14057.7186

PhoBERT 27 có 135.019.035 tham số, 28 có 135.019.804; BiGRU lần lượt 2.128.283 và 2.128.540. TF-IDF đếm hệ số và bias, không gồm IDF. CPU fit và GPU train có phạm vi đo khác nhau; thời gian không gồm preprocessing/download. Chi phí thực tế phụ thuộc batch, architecture và phần cứng.

9. Chuẩn bị bảo vệ giữa kỳ

9.1. Bài nói ngắn cho ba mô hình

TF-IDF fit vocabulary/IDF chỉ trên train rồi fit OvR LR cho từng nhãn; ngưỡng 0,5. Kết quả 27 Macro-F1 0,238825, 28 là 0,231040. Đây là baseline từ vựng, không dùng pretrained embedding.

BiGRU học embedding và trọng số từ đầu; hai hướng hidden 128 ghép thành 256, dropout rồi linear head. BCE/AdamW, batch 64, lr=0,001, đủ 20 epoch. Best 27 ở 17, 28 ở 20; Macro-F1 27=0,421976, 28=0,408615.

PhoBERT-base-v2 tải backbone pretrained, VnCoreNLP tách từ trước BPE, fine-tune toàn bộ backbone/head. BCE, AdamW lr=2e-5, batch 16, đủ 20 epoch. Best 27 ở 14 đạt 0,597410; best 28 ở 15 đạt 0,578341. Bản 28 chiếu 27 đạt 0,583580, chưa cao hơn bản 27.

9.2. Năm câu hỏi đề bài

Dùng các bảng mới trong reports/kaggle_equal20_2026-11-05/: label_extremes, frequency_f1, pair_preservation, neural_style_comparison, threshold_sensitivity. PhoBERT 27 cao nhất gratitude (0,8370), thấp nhất disapproval (0,3623); Spearman tần suất–F1 khoảng 0,018, p≈0,928. Trong các nhóm khảo sát, teencode có delta PhoBERT−GRU lớn nhất khoảng 22,13 điểm phần trăm. Tất cả là mô tả validation của study 27 mới, không chuyển số từ run cũ.

9.3. Câu hỏi bảo vệ thường gặp

Câu hỏi	Câu trả lời
Vì sao sigmoid thay softmax?	Multi-label không loại trừ; sigmoid từng logit không ép tổng xác suất bằng 1.
Fine-tuning xảy ra ở đâu?	model.parameters đi vào AdamW; mỗi batch backward, clip rồi optimizer.step cập nhật backbone và head.
Validation có học trọng số không?	eval + no_grad; không backward/step. Nó dùng để chọn checkpoint và sau này chọn decoder.
20 epoch nhưng best 14/15 là sao?	Train đủ 20; cuối cùng nạp checkpoint có emotion27 F1 cao nhất, không mặc định epoch cuối.
Epoch 1 F1=0 có hỏng model?	Chưa có true positive tại cutoff 0,5 ở epoch đầu; run 27 sau đó đạt 0,597410.
Neutral giúp cải thiện không?	Chưa thấy trên emotion27 trong lượt này: model 28 chiếu 27 thấp hơn model 27; một seed chưa đo ý nghĩa thống kê.
Có chạy cả hai T4 không?	Không; code dùng một GPU mỗi process, không cấu hình parallel training.
Hash khác có phải dữ liệu khác?	Đã chứng minh đúng biến thể byte LF/CRLF; parsed config, fixed IDs và nội dung local khớp.
Đã dùng test chưa?	Audit dữ liệu/trùng lặp có dùng thông tin split; không chấm model hoặc tune bằng test.
Split đã stratified chưa?	Audit không chứng minh thuật toán tạo split; còn xác nhận official split với giảng viên.

9.4. Phân tích lỗi và giới hạn

Sáu ca PhoBERT 28 mới: ba Neutral FP tự tin nhất, ba Neutral FN xác suất thấp nhất. Giữ ID/nhãn/xác suất và mô tả trừu tượng; không copy raw text. Nhận xét về ngữ cảnh, HTML, câu ngắn và từ tục là giả thuyết có hỗ trợ AI, cần nhóm xác nhận. 20 FN/20 FP tự sinh mỗi model là ứng viên để đọc, không phải audit hoàn tất.

Giới hạn chung: một seed, validation dùng chọn checkpoint/khảo sát ngưỡng, style rules có nhiễu, test chưa chấm. Bài nói theo lời dẫn20 slide. Bổ sung tên/MSSV/phân công trước buổi bảo vệ.

10. Tờ tra cứu và nguồn đối chiếu

10.1. Thuật ngữ và cấu hình

Khái niệm	Ý nghĩa trong lượt hiện tại
Logits / sigmoid	Điểm chưa giới hạn → xác suất độc lập từng nhãn
BCEWithLogitsLoss	Kết hợp sigmoid/BCE ổn định số trong training
Macro / Micro	Trung bình F1 theo nhãn / gộp TP,FP,FN mọi nhãn
LRAP	Xếp hạng nhãn thật từ xác suất, không phụ thuộc cutoff
Epoch / best	Neural đủ 20 ở 27/28; GRU best 17/20, PhoBERTbest 14/15
Head / backbone	Linear output mới / biểu diễn pretrained được fine-tune
Config / nguồn	configs/kaggle_equal_20ep.json; notebook 08/09 executed; hai ZIP equal20

10.2. Địa chỉ mã để mở khi được hỏi

scripts/train_neural.py:19–31 định nghĩa BiGRU; 34–86 preprocessing/tokenizer/loaders; 95–102 inference; 134–166 training/checkpoint. scripts/neutral_28_labels.py thêm output 28 và dùng selection_macro_f1 trên 27 cảm xúc. scripts/kaggle_equal_epochs.py điều phối profile, manifest, report và archive.

scripts/import_equal20_results.py kiểm chứng archive/notebook, metric NPZ, history và file variants. scripts/build_equal20_report.py tạo report/slide/Q1–Q5 từ evidence mới; build_report.py mặc định gọi builder này. scripts/build_study_guide.py kiểm tra source quotes và render PDF cẩm nang.

10.3. Tài liệu tham khảo

[1] Đề ViGoEmotions và student-guide.pdf trong docs/, yêu cầu Midterm A–E và Final RQ1.

[2] Hung et al., ViGoEmotions, EACL 2026, DOI 10.18653/v1/2026.eacl-long.129; điểm paper không phải điểm của nhóm.

[3] Nguyen và Nguyen, PhoBERT, EMNLP Findings 2020; model vinai/phobert-base-v2 và revision trong config.

[4] scikit-learn, PyTorch, Hugging Face Transformers và VnCoreNLP: phiên bản theo manifest của study. Nội dung thuật toán/metric giải thích theo source, số đo lấy từ NPZ/run JSON thực.

[5] Evidence chính: reports/kaggle_equal20_2026-11-05/, provenance.json và các checksum notebook/ZIP. Các artifact trước đây không được dùng làm nguồn điểm hiện tại.